

**UNIVERSITY OF PERADENIYA**

Department of Computer Engineering

# **Lessons Learnt from the Second Year Project**

An expository article about the experience gathered from CO2060 : Software Systems Design  
Project and recommendations for future enhancements

E/23/182 – Kulasooriya R K

September 2026

# My Experience Working on the Project

The second year project was one of the first opportunities where I had to work on a software system that was meant to solve a real problem instead of only completing an academic exercise. Our project, the *Orthodontics Workflow Automation System*, was developed to support work in an orthodontic clinic. It includes patient records, appointments, queue management, clinical notes, student case supervision, inventory management, reports, and user access control.

At the beginning, the project looked like a normal software development task. However, after meeting stakeholders and receiving feedback during demonstrations, I understood that building a system for real users requires much more than writing code. The users care about whether the system is easy to understand, whether information is correct, whether they can complete a task quickly, and whether patient information is safe. These practical concerns changed the way I thought about software development.

One important lesson I learnt was the value of understanding the workflow before implementing features. In the early stages, it was easy to think about the project in terms of pages, buttons, and database tables. Later, we had to focus more on how reception staff, nurses, orthodontists, dental surgeons, students, and administrators would actually use the system. For example, a receptionist should be able to register a patient and request clinical assignments, while an orthodontist should approve relevant requests and supervise student work. This showed me that a feature is not complete just because it works technically. It should also fit naturally into the user's daily work.

I also learnt that user roles and permissions need careful planning. In a healthcare-related system, not every user should see or change all information. We had to consider which roles could create patients, manage appointments, update clinical information, access payments, manage inventory, or view audit logs. Small mistakes in permission rules can create privacy or security problems. Through this work, I understood why role-based access control is important and why both the frontend and backend must enforce it. Hiding a button in the user interface is helpful, but the backend must still reject unauthorized requests.

Another lesson was the importance of communication within the team. Different

members worked on different parts of the system, such as frontend interfaces, backend APIs, database design, testing, deployment, and documentation. Therefore, we had to regularly discuss changes and make sure that one part of the system did not break another part. For example, if the backend response format changed, the frontend needed to be updated accordingly. I learnt that maintaining clear communication, sharing progress, and reviewing each other's work saves time later.

Testing was also a major learning experience. During demonstrations, even small issues became visible quickly. A missing validation message, an unclear button label, or an incorrect role restriction could affect the user's confidence in the system. We learnt to test not only the successful path, but also cases such as invalid inputs, inactive users, failed login attempts, missing assignments, deleted records, and unavailable services. This made me realize that testing is not a final step done after development. It should happen throughout the project.

Finally, I learnt that documentation is part of the product. At first, documentation may seem less important than implementation. However, when the project is handed over to staff, instructors, or future batches, they need to understand how accounts are created, how roles work, how features should be used, and how the system can be deployed or maintained. Good documentation helps the next person continue the work without depending fully on the original development team.

## **Recommendations**

### **For Staff and Instructors**

Staff members should continue providing feedback based on real clinic work. Feedback from demonstrations was valuable because it helped us understand details that are difficult to identify from requirements alone. Before using the system fully, staff should agree on clear procedures for account creation, patient assignment, appointment handling, and clinical record updates.

It is also recommended to identify one or two responsible administrators for user management and basic system maintenance. They should regularly review inactive

accounts, reset passwords when necessary, monitor audit logs, and ensure that patient information is handled carefully. Regular backups of the database and uploaded files should also be maintained before important system changes or deployments.

Short training sessions would be useful for each user role. Reception staff, nurses, clinicians, students, and administrators do not use the same features. Role-specific training can reduce mistakes and help users become comfortable with the system faster.

## **For Future Batches**

Future batches should begin by studying the existing documentation, project structure, database schema, and user roles before adding new features. They should avoid changing the system without understanding the current workflow, because a small change in one module can affect several roles or records.

The next team could improve the project by adding more automated tests, improving mobile usability, strengthening reporting features, and collecting user feedback after longer periods of real use. They may also consider notification improvements, better dashboard summaries, and more detailed backup and recovery procedures.

Most importantly, future batches should keep the documentation updated whenever they change a feature. The system will be easier to maintain if the code, user guides, deployment instructions, and permission descriptions remain consistent. This project has given us a useful foundation, and careful future work can make it more reliable and more useful for the orthodontic clinic.